Design Project

Automatic Car Washing System

EEX 4436

Microprocessor and Interfacing

by

P.G.K.N. Kulathilake

623645987

Submitted to

Department of Electrical and Computer Engineering

Faculty of Engineering Technology

The Open University of Sri Lanka

at

Colombo Regional Center

on

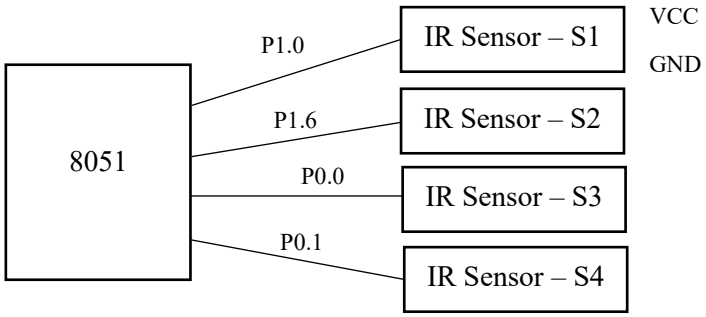
04th December 2025

Q1] For the proposed mechanism, identify suitable sensors and actuators to detect the following and draw simple block/interfacing diagrams for each to explain your methodology.

a. to detect vehicle presence at four stages

IR photoelectric sensors can be used for this.

When a car presents, the IR rays emitted by the sensor will be reflected back and the sensor will detect the reflected rays. When there's no any car present, it won't receive reflected IR rays. From this it is able to detect the presence of a car.

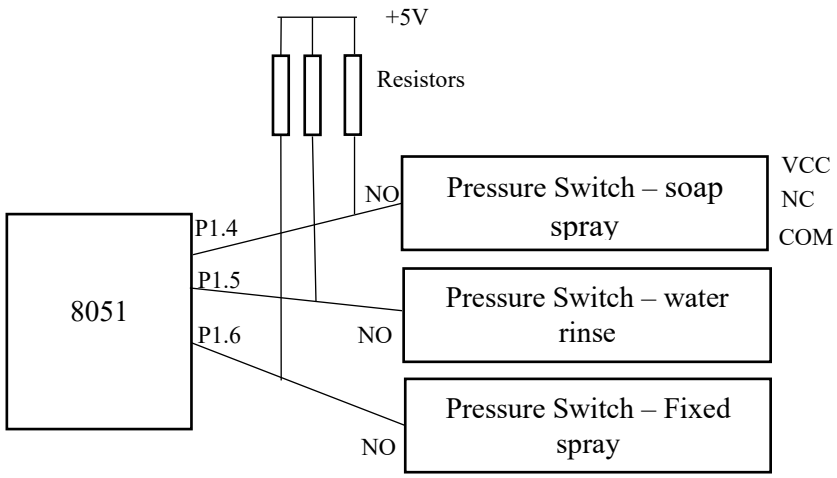


Four sensors will be interfaced with P1.0, P1.6, P0.0 and P0.1 pins of the MCU at the four stages (S1-S4) respectively.

Sensor	Actuator	Assumptions
IR Sensor (S1 to S4)	Conveyor Motor (M)	IR sensors with a sensing range of around 0.5 m is sufficient for this application.

		The signals for controlling conveyor during transitions of stages are generated based on the inputs of these IR sensors.

b. To monitor spray nozzle pressures.



The pressure switch can be used to monitor the pressure of the spraying water. When the pressure of the spraying water is crossing a threshold value, the pressure switch will be closed. Then the input on the pin where the particular pressure switch is connected will be changed and based on that we can take an action to stop the system and to rectify that fault. When the pressure switch activates, the NO pin will be connected with COM which is connected to ground. It will make the MCU pin low. In this manner it is able to detect if pressure goes below 60 psi

In the simulation if the pressure switch activates, the system will go in to emergency stop state.

Sensor	Actuators	Assumptions
Pressure Switch	Solenoid Valve to stop spraying	The pressure switch can be configured to the required threshold point. At that threshold pressure, its NO pin will be connected to ground making the input on the MCU connected pin 0. Based on that signal, it is able to turn off the solenoid valve in order to stop spraying and alarm about the issue.

c. To ensure vehicle weight doesn't exceed 2000 kg

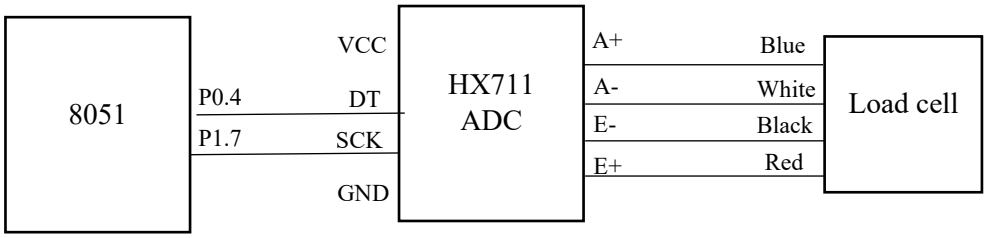
A load cell with the HX711 ADC module can be used to ensure if the vehicle weight doesn't exceed 2000 kg. The weight of the car can be measured from this and can decide if the process should continue only if the weight is not exceeding 2000 kg

The HX711 is a 24 bit ADC. Therefore it can provide weight readings taken from the load cell more accurately with higher resolution. It is able to read the binary output of 24 bits corresponding to the weight serially using one pin. Therefore weight measurement can be read using only two pins including one for sending clock signal to read bit by bit serially.

In order to show that value on LCD, it has to be converted to decimal. As 8051 is supporting only 8 bit arithmetic operations, it would be complex to do that conversion. Also it requires three register to store the binary value read.

Therefore in the simulation the weight was displayed considering only the lower byte of the 24 bit value. It was assumed that there is a person operating this system and he has to start the process manually by pressing start button after reading the weight on display if it is not exceeding the maximum value.

This weight sensor has to be calibrated by measuring a known weight. We can then add or subtract a value in order to compensate for the offset.

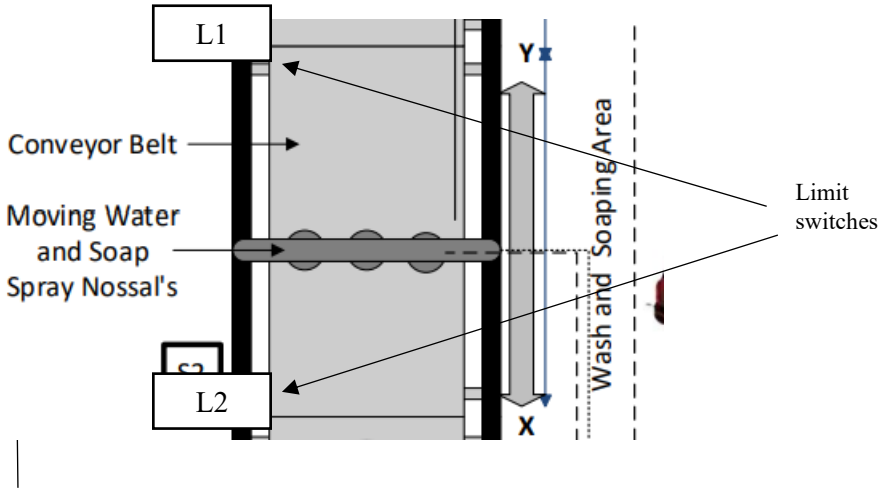


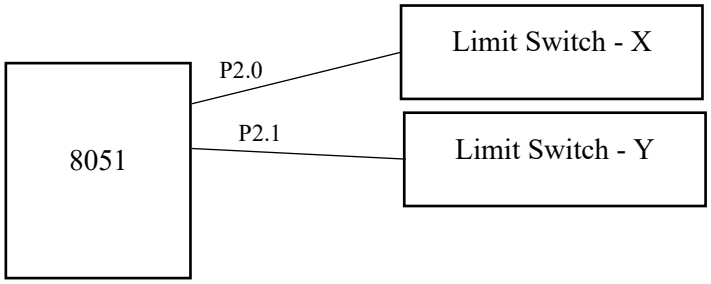
Sensor	Actuators	Assumptions
Load cell	Conveyor Motor (M)	If the value taken from load cell lies within safe margin, the process can be started automatically by moving the conveyor. But in the simulation, it was assumed that once the weight is displayed on the LCD, the operator has to manually start the process by pressing “start” if the weight is not exceeding the safe margin. This manual starting process was employed as it is bit complex to convert the 24 bit weight measurement value received from the ADC used in the simulation

		in to decimal for automatic comparison.

d. To move spray nozzles forward and backward within x-y area only

Two limit switches can be used to move the spray nozzles only within the X-Y area. The two switches can be placed on X and Y. When the spray nozzles hit on any of those limit switches, the motors which move the nozzle in that direction can be stopped and change its direction to move backwards again.





Sensor	Actuators	Assumptions
Limit Switch	Motor that runs spray nozzles linearly in XY path	When the moving spraying mechanism moves and hits on any of the limit switches, it can be detected by MCU through the pin which limit switch connects to the MCU. Based on that signal the rotating direction of the motor that drives the spray mechanism can be changed through the motor driver.

Q2. a. Analyze the given specifications/parameters related to the operations/processes of the ACWS and then prepare a table of resources/features and conditions/constraints required to design the ACWS. (Table should contain Processes, Sub-Operations, Parameters, Conditions, Resources, Feature Required of 8051 microcontrollers, and Comments).

Process	**Sub Operations**	**Parameters**	**Conditions**	**Resources**	**Features of 8051 micro controller**	**Comments**
Entry	Check if vehicle is present	S1	if S1 is high then continue	IR sensor	P1.0 – S1	
	Check the weight of the car	Weight of the car	If weight of the car is less than 2000 kg, then continue	Load cell and HX711 ADC	P0.4 - DT P1.7 - SCK	If the weight exceeds, it has to be warned and stop the process in the design. The value of weight will be displayed on LCD when a vehicle

						is detected in simulation. As converting a 24 bit binary value from ADC to decimal and displaying it on LCD is bit complex on 8051 with ALP, only the lower byte of the weight reading is taken in to consideration in the simulation. It was assumed that there will be a person to start the process manually by pressing start button after reading the weight value displayed on the LCD.
	Wait for 5 seconds				Timer	
	Move the conveyor	25% D PWM	Run the motor until s1=0 and s2=1	Motor of conveyor belt (M) Motor driver Yellow LED Indicator	Timer P1.5 – ENA P1.3 – IN1 P1.4 – IN2	When the car reaches to the stage 2 the conveyor has to be stopped The yellow indicator will represent the transitioning between stages
Pre wash Phase	Move the spray		If a limit switch is	Limit switch – X	P3.5 – Limit Switches	If the pressure switch activates, the process

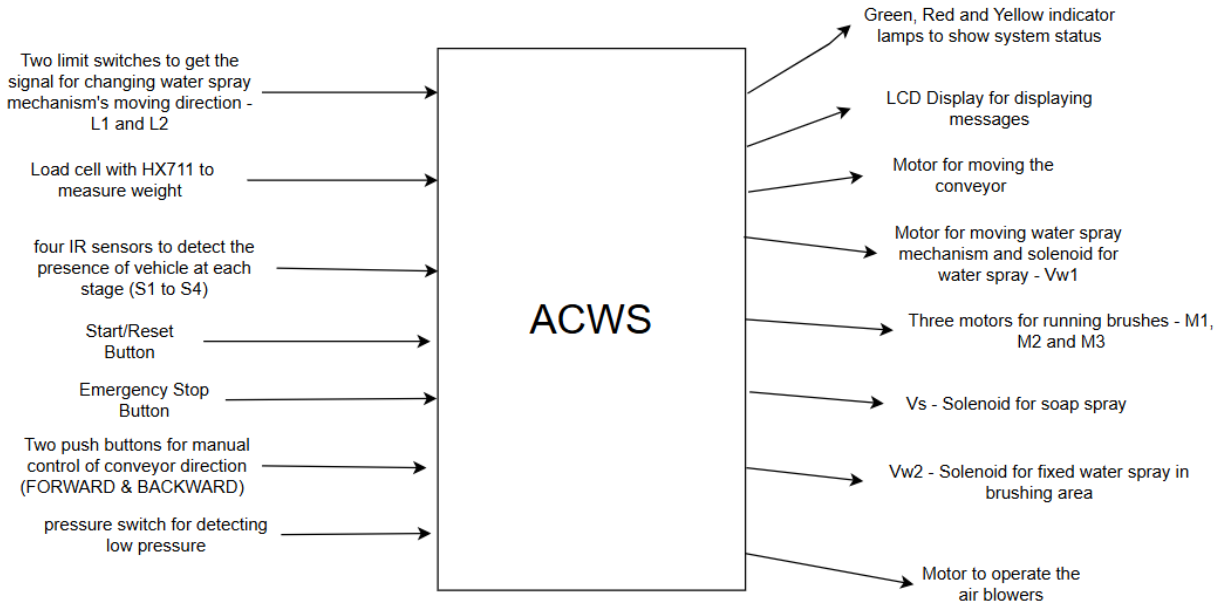
	mechanism front to back and back to front		hit change the direction of the spray mechanism	Limit switch – Y Motor for spray mechanism drive Decoder 4555	P0.6 – Decoder_A P0.7 – Decoder_B P0.3 – Decoder_EN	has to be stopped and alarmed. It is considered as the solenoid Vw1 also powers with the motor that moves the spray mechanism
	Water rinse	For 15 s		Solenoid – Vw1	Vw1	It is assumed that this Vw1 solenoid is opened until the spray mechanism runs
	Spray soap	For 15 s		Solenoid – Vs	P0.6 – Decoder_A P0.7 – Decoder_B P0.3 – Decoder_EN	Second output of decoder activates soap spray solenoid
	Move the conveyor	25% D PWM	Run the motor until s2=0 and s3=1	Motor of conveyor belt (M) Motor driver Yellow LED Indicator	Timer P1.5 – ENA P1.3 – IN1 P1.4 – IN2	When the car reaches to the stage 3 the conveyor has to be stopped The yellow indicator will represent the transitioning between stages
Mechanical Brushing	Run scrub motors	120 rpm	Run for 10s	Motors M1, M2 and M3	P0.6 – Decoder_A	All three motors will be controlled simultaneously from the same signal. This

					P0.7 – Decoder_B P0.3 – Decoder_EN	signal is sent from the third output of the decoder. It was assumed that the motor runs at 120 rpm when it is provided with full voltage without PWM.
High pressure rinsing	Deliver a high pressure rinse	For 10 s		Solenoid - Vw2	P3.4	Assumed that high pressure rinsing will start after brushing and it will continue for 10s
	Move the conveyor	25% D PWM	Run the motor until s3=0 and s4=1	Motor of conveyor belt (M) Motor driver Yellow LED Indicator	Timer P1.5 – ENA P1.3 – IN1 P1.4 – IN2	When the car reaches to the stage 4 the conveyor has to be stopped The yellow indicator will represent the transitioning between stages
Drying cycle	Activate air blowers	75% D PWM	Run for 15s or until S4 =0	Air Blowers	P3.0	Air blowers will run until the vehicle exits from the last stage or otherwise it will be turned off after 15 s
Emergency Stop	Triggering Emergency Stop		Manual Stop with emergency stop button	Emergency Stop button Red Light Indicator	P3.6 P3.7 P0.5	The emergency stop will be activated whenever the Emergency stop

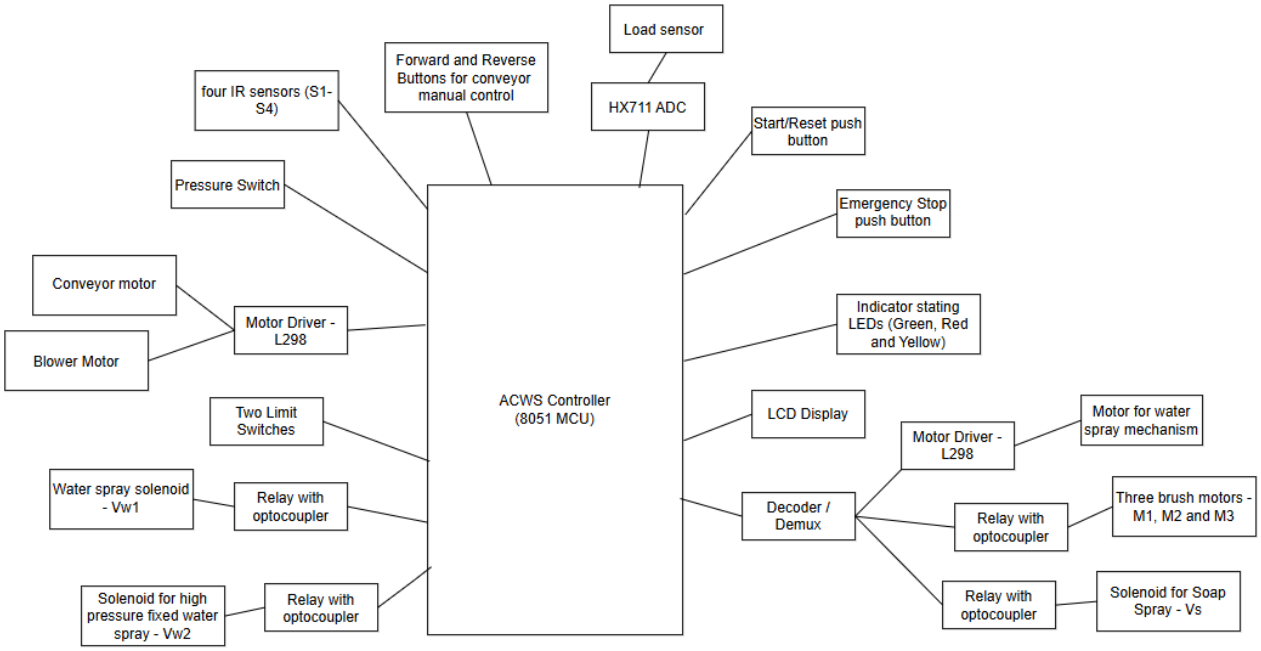
						button is pressed manually The Red light indicator will be glowing when this is activated
	Manual Control of the conveyor	25% D PWM	Move forward if FWD button pressed Move backwards if REV button is pressed	FWD button REV button Conveyor motor	FWD – P3.1 REV – P0..2	The system will halt when emergency stop is activated. The conveyor can be controlled manually at this stage using the two buttons FWD and REV.
	Resetting the system		Reset the system if RESET button is pressed	Reset button	RESET BUTTON – P3.6 RESET MCU – P3.7	When the RESET button is pressed, a message will be shown that system is going to reset and after a delay, the system will start again from beginning. The pin 3.7 is used to provide a triggering pulse for RST pin on microcontroller to reset the system
Halt the system during the detection of an	Triggering Pressure Switch	< 60 psi	Trigger if pressure switch identifies a pressure	Pressure Switch	INTR1_ISR	If an abnormal pressure is detected by the pressure switch, it will trigger the INTR1_ISR as the

abnormal pressure			above 60 psi			pressure switch connects with the external interrupt 1 pin.
	Save current status of the actuators			Decoder 4555 Conveyor Motor Brushes Blower Solenoid valves	Bit addressable memory area in RAM	This will save the status of all the actuators in to the bit addressable area of the RAM from 20H.1 to 20H.4
	Resetting the system			Reset button	P3.6	Once the fault is rectified related to the abnormal pressure, the system can be continued from where it was stopped by pressing this reset button. Before resetting, it will write the last status of the actuators so that process doesn't disturbed.

b. Draw the external view of the system, i.e., a diagram that shows the inputs/sensors and the outputs/actuators of the system.



c. Identify the subunits/sub-modules of the system and draw the interconnected block diagram of the system using the central controller and other required interfacing devices.

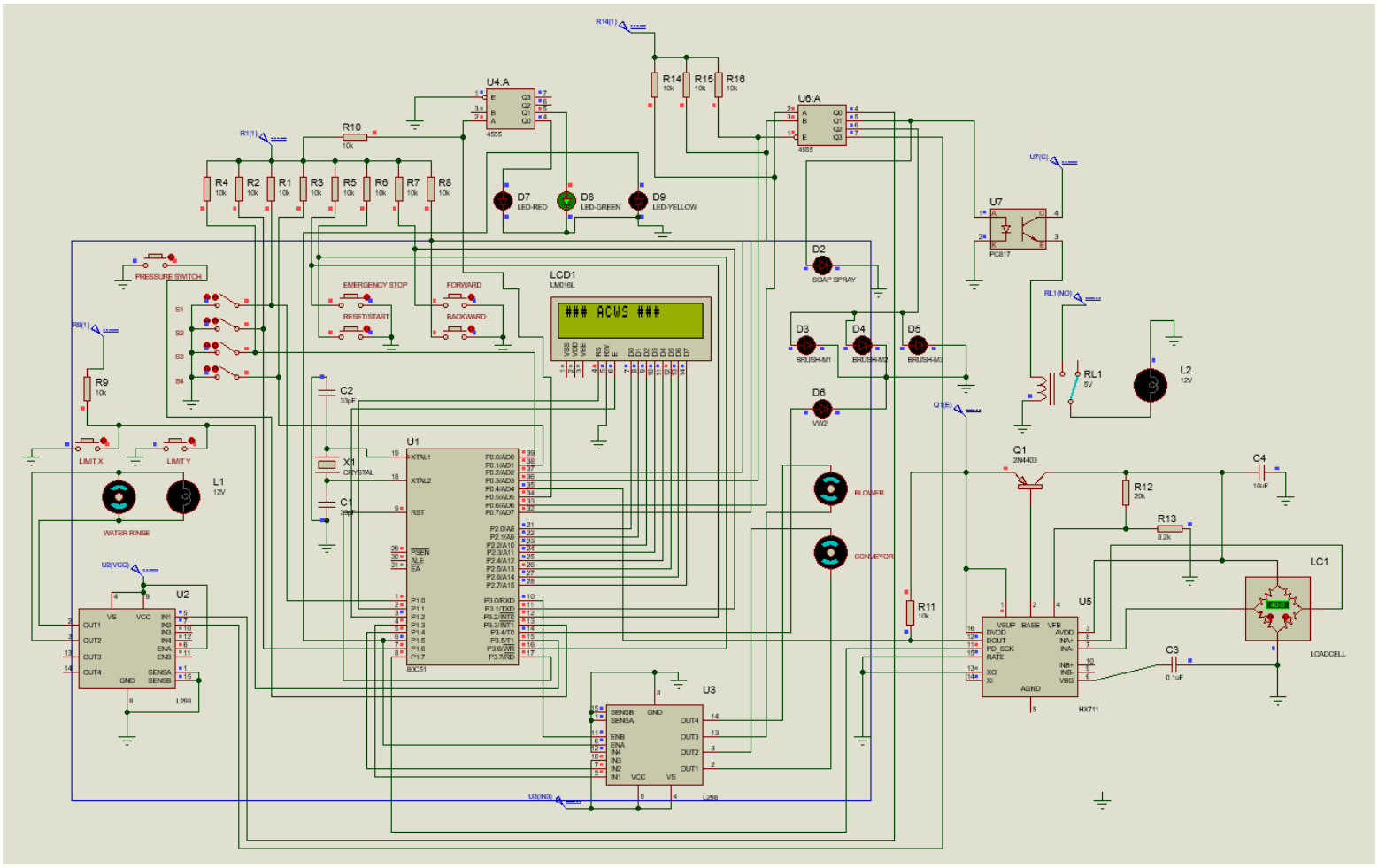


[Q3] Draw a schematic diagram of the system. Clearly show each electronic components/module. Use any suitable software tools to create a schematic diagram.

Port Map

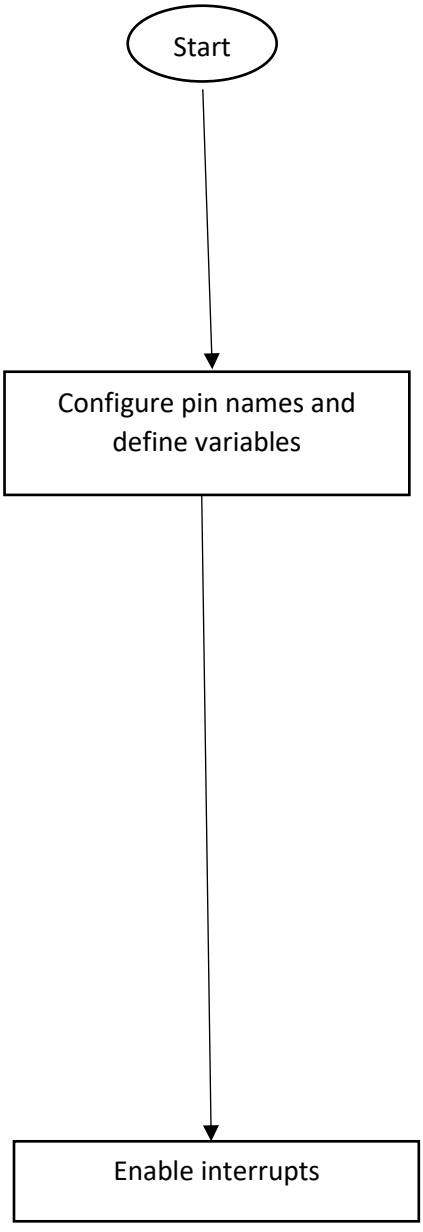
Pin	**Component**
P0.0	S3 IR Sensor
P0.1	S4 IR Sensor
P0.2	REV Button
P0.3	Decoder_EN
P0.4	DOUT of HX711 ADC
P0.5	LED_R / LED_G
P0.6	Decoder_A
P0.7	Decoder_B
P1.0	S1 IR Sensor
P1.1	LCD_RS

P1.2	LCD_EN
P1.3	IN1 - Conveyor
P1.4	IN2 - Conveyor
P1.5	ENA - Conveyor
P1.6	S2 IR Sensor
P1.7	SCK of HX711 ADC
P2.0	LCD D0
P2.1	LCD D1
P2.2	LCD D2
P2.3	LCD D3
P2.4	LCD D4
P2.5	LCD D5
P2.6	LCD D6
P2.7	LCD D7
P3.0	BLOWER_ENB
P3.1	FWD Conveyor
P3.2	Emergency Stop
P3.3	Pressure Switch
P3.4	VW2 Spray nozzle solenoid
P3.5	Limit Switches
P3.6	RESET BUTTON
P3.7	RESET MCU

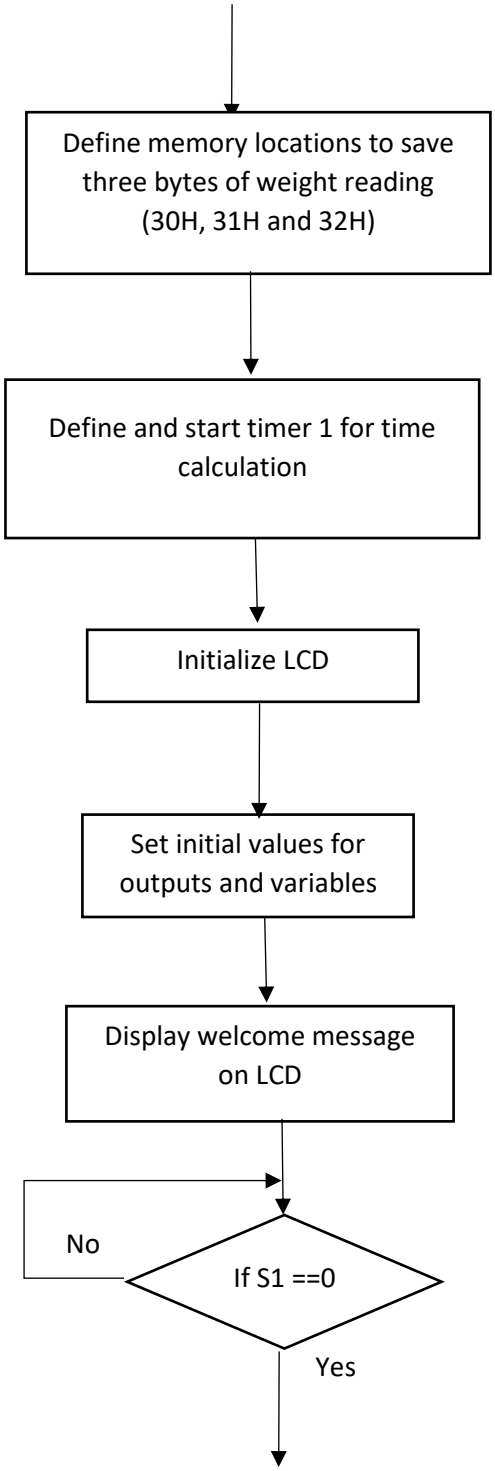

R14(1)
R14
R15
R16
U4:A
U6:A
R1(1)
R10
R4
R2
R1
R3
R5
R6
R7
R8
D7
LED-RED
D8
LED-GREEN
D9
LED-YELLOW
U7(C)
U7
PC817
D2
SOAP SPRAY
PRESSURE SWITCH
EMERGENCY STOP
FORWARD
RESET/START
BACKWARD
LCD1
LM016L
ACWS
RS(1)
R9
S1
S2
S3
S4
D3
D4
D5
BRUSH-M1
BRUSH-M2
BRUSH-M3
RL1(NO)
RL1
5V
L2
12V
D6
VW2
C2
33pF
X1
CRYSTAL
C1
33pF
U1
80C51
Q1(E)
Q1
2N4403
C4
10uF
R12
20k
R13
8.2k
LIMIT X
LIMIT Y
L1
12V
WATER RINSE
BLOWER
CONVEYOR
LC1
LOADCELL
U2(VCC)
U2
L298
R11
10k
U5
HX711
C3
0.1uF
U3
L298
U3(IN3)

[Q4] Draw flowchart(s) to represent the algorithm of the ACWS operation. Clearly show the port mapping of the 8051 microcontrollers. (i.e. Mark the pin connection of each sensor and actuators used in the ACWS)

Flow Chart for the Main Program Flow



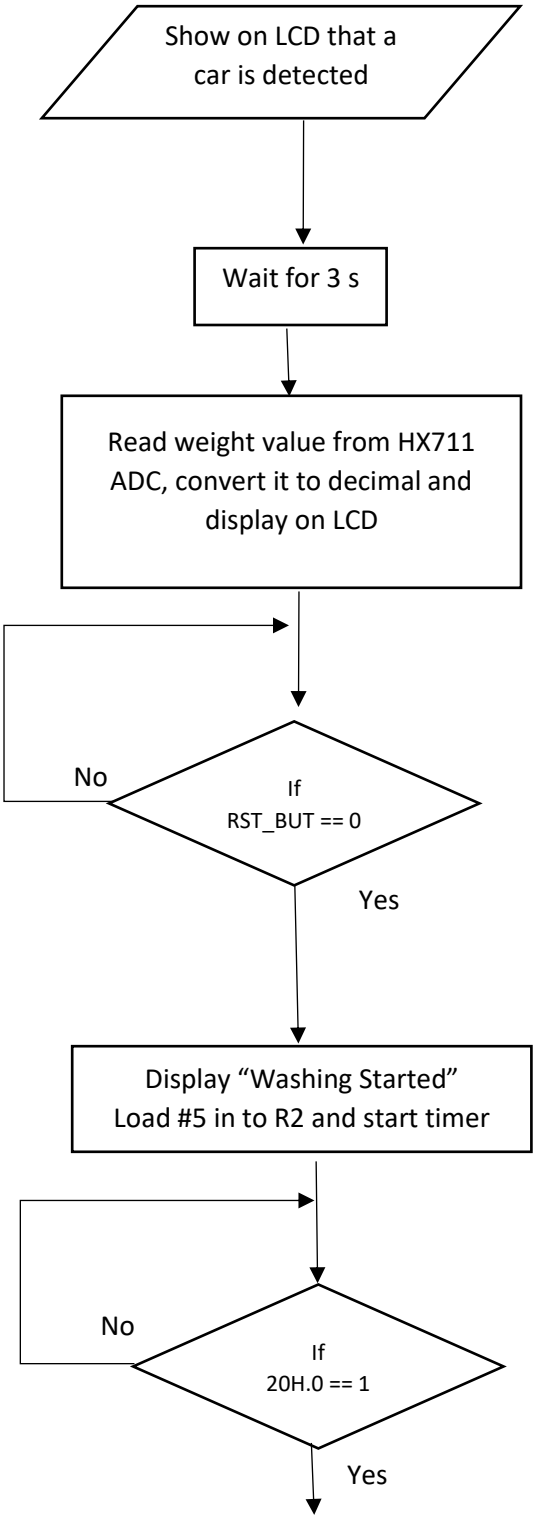
ORG 0000H	
S1 BIT P1.0 LCD_RS BIT P1.1 LCD_EN BIT P1.2 IN1 BIT P1.3 IN2 BIT P1.4 ENA BIT P1.5 S2 BIT P1.6 VW1 BIT P1.7 S3 BIT P0.0 MUL_A BIT P0.6 MUL_B BIT P0.7 MUL_EN BIT P0.3 LCD_DATA DATA P2 BLOWER_ENB BIT P3.0 S4 BIT P0.1 STOP BIT P3.2 FWD BIT P3.1 REV BIT P0.2 LED_R BIT P0.5 RST_BUT BIT P3.6 RST_MCU BIT P3.7 DT BIT P0.4 SCK BIT P1.7 LIMIT BIT P3.5 VW2 BIT P3.4	S1, S2, S3 & S4 – IR Sensors at 4 stages LCD pins → LCD_EN & LCD_RS Port for LCD data → LCD_DATA Conveyor → IN1, IN2 & ENA Decoder for water rinse, soap spray and brush motors → MUL_A, MUL_B & MUL_EN Blower otor → BLOWER_ENB Emergency stop button – STOP Vw2 – fixed water spray
MAIN: SETB EA SETB EX1 SETB IT1 SETB EX0 SETB IT0 SETB ET0	Enabling global interrupts, exteral interrupts and timer interrupts



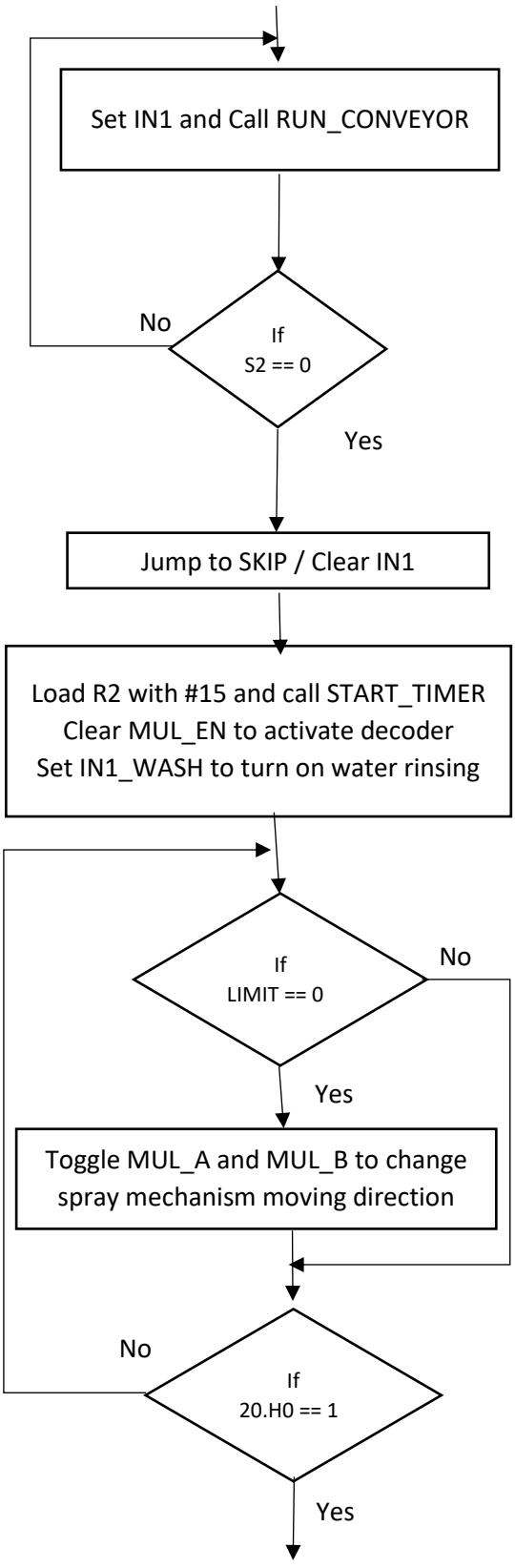
HIGH_BYTE EQU 30H MID_BYTE EQU 31H LOW_BYTE EQU 32H	These memory location are used to store 24 serial bits from hx711 adc

MOV TH1,#04CH
MOV TL1,#000H
SETB TR1
	Using timer 1 to calculate times

ACALL LCD_INIT MOV A,#080H ACALL LCD_CMD	Initializing LCD
CLR BLOWER_ENB SETB MUL_EN CLR ENA SETB REV SETB LED_R CLR 20H.0 CLR VW2	
mov dptr,#STR_menu ACALL PRINT	
JB S1,$	If a car is detected the process will be proceeded



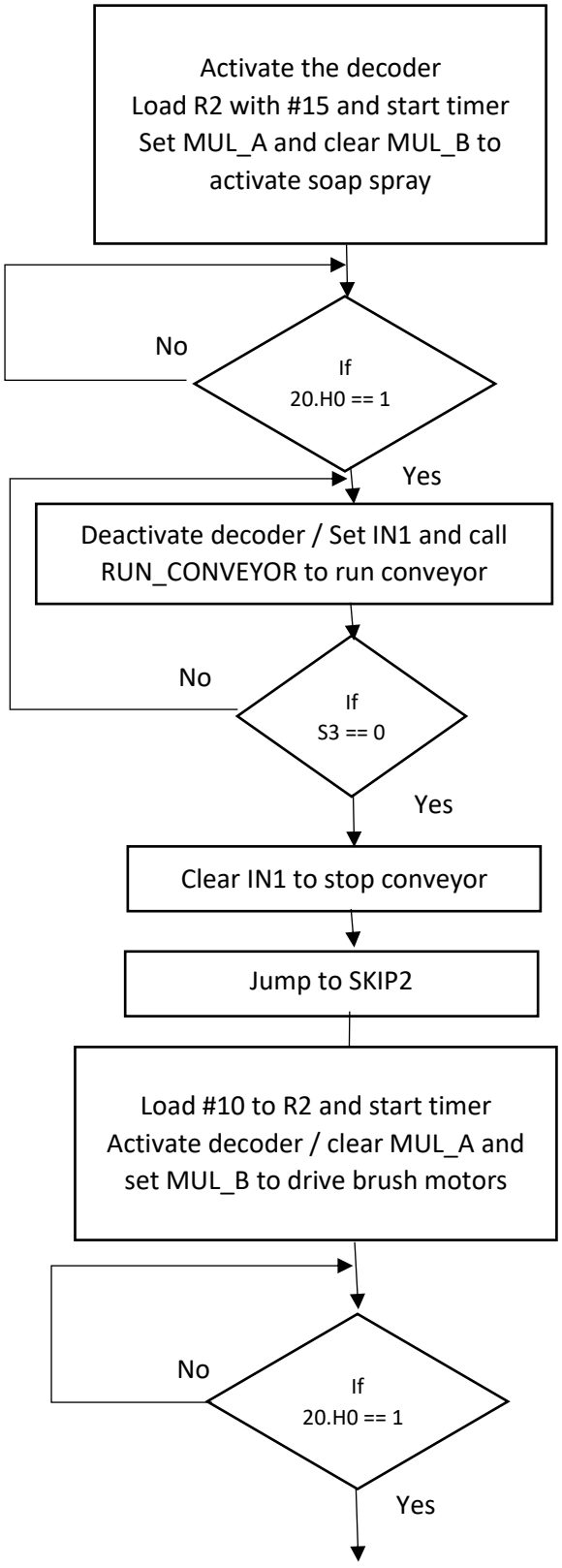
ACALL LCD_CLR MOV DPTR,#MYSTRING ACALL PRINT	
ACALL DELAY_3S	
ACALL DISP_WEIGHT	In the code, this is included as a part of the main program. For simplicity, here it is denoted as a subroutine named DISP_WEIGHT
JNB RST_BUT, MOVE_NEXT SJMP LOOP_WEIGHT	It was assumed a user will manually check the weight and press start button to start the process if weight is in acceptable range
MOVE_NEXT: ACALL LCD_CLR MOV A,#080H ACALL LCD_CMD MOV DPTR,#MYSTRING1 ACALL PRINT	
;waiting for 5 seconds MOV R2,#5 ACALL START_TIMER JNB 20H.0,$	Finishing of timer counting is determined by the status of the first bit in 20H byte which is in the bit addressable area. This is used in each timer measurements.



;running conveyor until it goes to stage 2
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#MYSTRING2
ACALL PRINT
SETB IN1
CLR IN2
GO:ACALL RUN_CONVEYOR
JNB S2,SKIP
SJMP GO
	The conveyor will run until vehicle is detected at S2

SKIP: CLR MUL_EN	

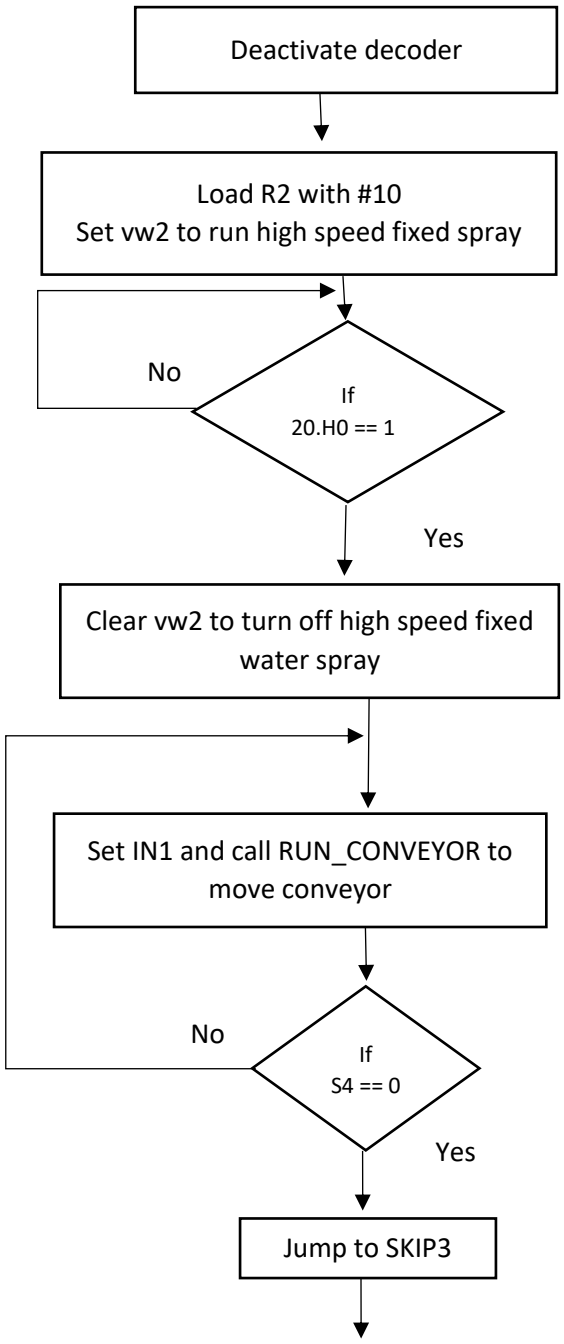
MOV R2,#15
ACALL START_TIMER
;SETB IN1_WASH
;CLR IN2_WASH
CLR MUL_A
CLR MUL_B
ACALL LCD_CLR
MOV DPTR,#STRING_3
ACALL PRINT
 	
CHK1:
JB LIMIT,CHK2
ACALL DELAY_1MS
JNB LIMIT,CHK
SJMP CHK2
 	
CHK:CPL MUL_A
CPL MUL_B
CHK2:
JNB 20H.0,CHK1
 	
SETB MUL_EN
	Whenever a limit switch is hit by the spray mechanism , the direction of that will be changed by changing the direction of rotation of the motor driving it.




```
;soap spraying for 15 seconds

CLR MUL_EN
SETB MUL_A
CLR MUL_B
MOV R2,#15
ACALL START_TIMER
ACALL LCD_CLR
MOV DPTR,#STRING_4
ACALL PRINT

JNB 20H.0,$
SETB MUL_EN
```

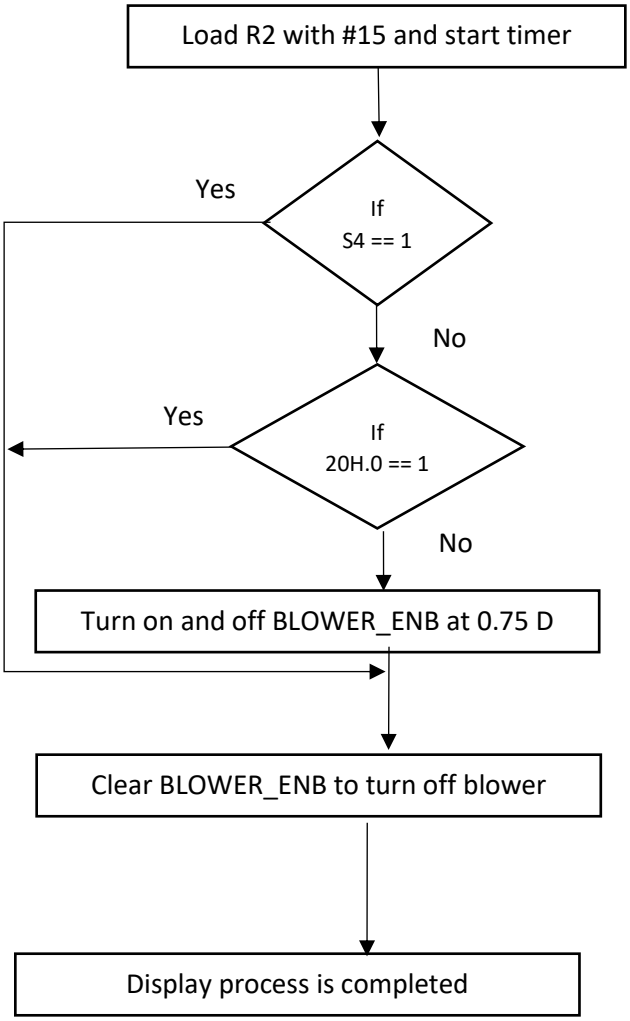
```
;running conveyor until car is detected at stage 3
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#MYSTRING2
ACALL PRINT
SETB IN1
CLR IN2
GO1:ACALL RUN_CONVEYOR
JNB S3,SKIP2
SJMP GO1
```

```
;starting brushing motors for 10 seconds
SKIP2:
OK:MOV R2,#10
ACALL START_TIMER
CLR MUL_EN
CLR MUL_A
SETB MUL_B
mov dptr,#STRING_5
ACALL LCD_CLR
MOV A,#080H
acall print
JNB 20H.0,$
SETB MUL_EN
```



MOV R2,#10
ACALL START_TIMER
SETB VW2
mov dptr,#STRING_hp
ACALL LCD_CLR
MOV A,#080H
acall print
 	
JNB 20H.0,$
 	
CLR VW2	

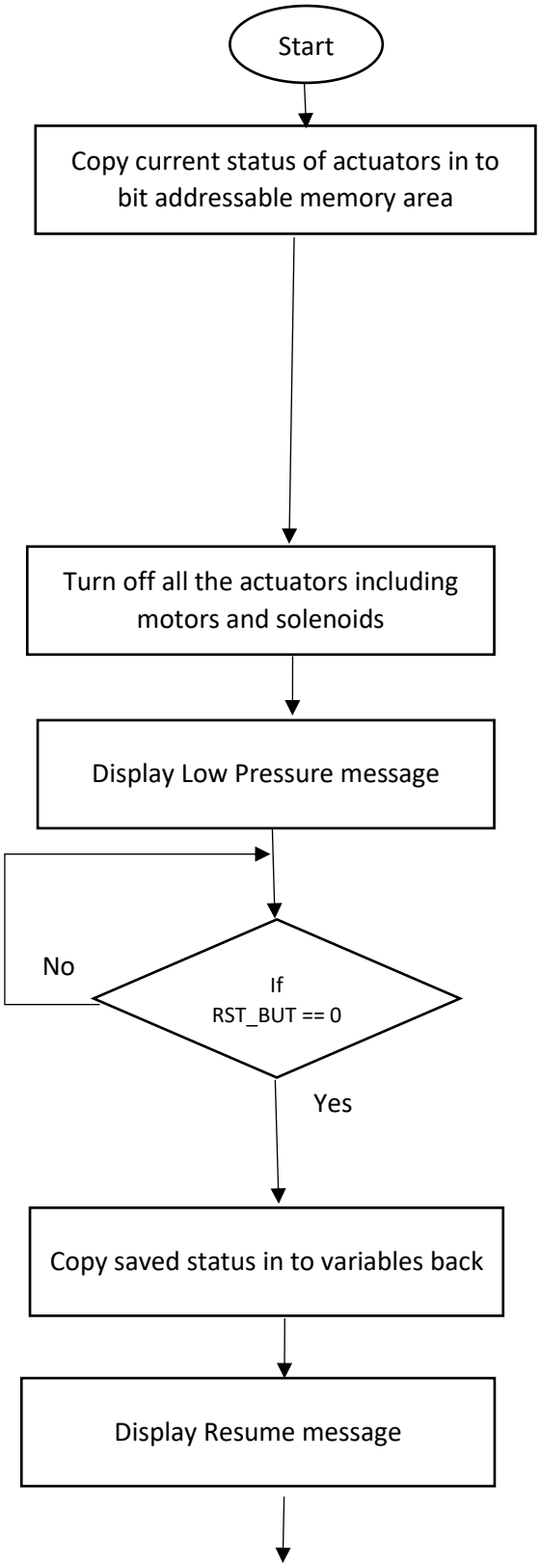
;running conveyor until car is detected at stage 4
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#MYSTRING2
ACALL PRINT
SETB IN1
CLR IN2
GO2:ACALL RUN_CONVEYOR
JNB S4,SKIP3
SJMP GO2	




```
;run blower for 15 seconds or until the car moves out of the stage 4
SKIP3:
MOV R2,#15
ACALL START_TIMER
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#STRING_6
ACALL PRINT

X11:
JB S4,EXIT11
JB 20H.0,EXIT11
CLR BLOWER_ENB
MOV R6,#75
DJNZ R6,$
SETB BLOWER_ENB
MOV R6,#25
DJNZ R6,$
SJMP X11
```

```
;process is completed
EXIT11:
CLR BLOWER_ENB
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#STRING_7
ACALL PRINT
```

ISR_INTR1 for Low Pressure Detecting



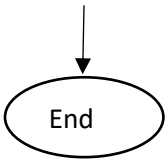

```
;interrupt to detect abnormal pressure

INTR1_ISR:

MOV C ,MUL_EN
MOV 20H.1,C
MOV C ,VW2
MOV 20H.2,C
MOV C,ENA
MOV 20H.3,C
MOV C ,BLOWER_ENB
MOV 20H.4,C

CLR LED_R

SETB MUL_EN
CLR VW2
CLR ENA
CLR BLOWER_ENB

ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
mov dptr,#STR_P
ACALL PRINT

POINT:JB RST_BUT,$
ACALL DELAY_1MS
JNB RST_BUT,OUT
SJMP POINT

OUT:
MOV C ,20H.1H
MOV MUL_EN,C
MOV C ,20H.2
MOV VW2,C
MOV C ,20H.3
MOV ENA,C
MOV C ,20H.4
MOV BLOWER_ENB,C

SETB LED_R

ACALL LCD_CLR
MOV A,#080H
```

The pressure switch is connected as an input to the MCU INT1 pin.

When a low pressure is detected, the MCU can sense that through the connected pin.

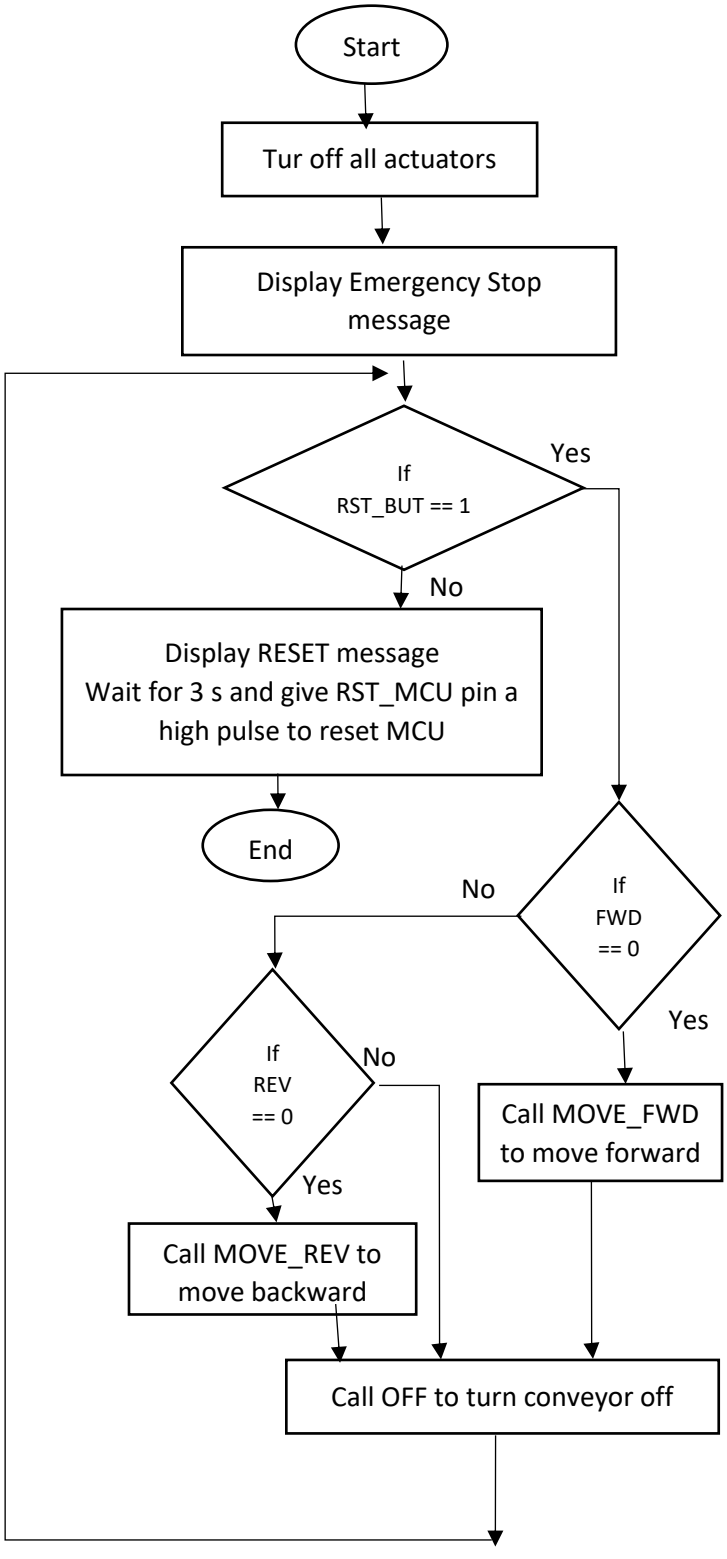
As edge triggered interrupts are enabled, a falling edge on INT1 pin will call the routine at ISR address at 0013H.

This ISR is written at that location and whenever this is triggered, the CPU will stop executing its current tasks and will complete this program giving it priority. Once this is completed it will go back to where it was executing finally.



ACALL LCD_CMD mov dptr,#STR_RESUME ACALL PRINT RETI	

ISR_INTR0 for Emergency stop and manual conveyor control



;interrupt to handle emergency stop and moving the conveyor
;backward and forward manually during the emergency stop
INTR0_ISR:
 	
CLR BLOWER_ENB
CLR ENA
CLR IN1
CLR IN2
CLR LED_R
 	
MOV A,#080H
ACALL LCD_CMD
mov dptr,#STR_EMERG
ACALL PRINT
 	
LOOP2:
JB RST_BUT,PROCEED
MOV A,#080H
ACALL LCD_CMD
mov dptr,#STR_RESET
ACALL PRINT
 	
ACALL DELAY_3S
CLR RST_MCU
MOV R7,#60
DJNZ R7,$
SETB RST_MCU
RETI
 	
PROCEED:JNB FWD,MOVE_FWD
JNB REV,MOVE_REV
JB FWD,OFF
JB REV,OFF
SJMP LOOP2
	When the emergency stop button is pressed, it will make the INT0 pin low which the stop button is connected.
	
	It will trigger the ISR_INTR0 which is written at ISR memory location 0003H.
	
	It will indicate the emergency stop state via LCD display and red light indicator. And the system will wait halted turning off all the actuators until the reset button is pressed to start the process again.
	
	Also in this mode the conveyor can be controlled manually to take out a vehicle which was inside when emergency stop was triggered

```
MOVE_FWD:
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#STR_FWD
ACALL PRINT
SETB ENA
SETB IN1
CLR IN2
SJMP LOOP2

MOVE_REV:
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#STR_REV
ACALL PRINT
SETB ENA
CLR IN1
SETB IN2
SJMP LOOP2

OFF:CLR ENA
CLR IN1
CLR IN2
MOV A,#080H
ACALL LCD_CMD
mov dptr,#STR_EMERG
ACALL PRINT
MOV A,#0C0H
ACALL LCD_CMD
MOV DPTR,#STR_BLANK
ACALL PRINT
SJMP LOOP2

RETI
```

[Q5] Write assembly language programs (ACWS to perform the above task. (i.e. Q4). Clearly show the assembly language routines with comments and relation with the flowcharts drawn in above Q4.

```
S1 BIT P1.0
LCD_RS BIT P1.1
LCD_EN BIT P1.2
IN1 BIT P1.3
IN2 BIT P1.4
ENA BIT P1.5
S2 BIT P1.6
VW1 BIT P1.7
S3 BIT P0.0
MUL_A BIT P0.6
MUL_B BIT P0.7
MUL_EN BIT P0.3
LCD_DATA DATA P2
BLOWER_ENB BIT P3.0
S4 BIT P0.1
STOP BIT P3.2
FWD BIT P3.1
REV BIT P0.2
LED_R BIT P0.5
RST_BUT BIT P3.6
RST_MCU BIT P3.7
DT BIT P0.4
SCK BIT P1.7
LIMIT BIT P3.5
VW2 BIT P3.4

ORG 0000H
LJMP MAIN

ORG 0003H
LJMP INTR0_ISR

ORG 000BH
LJMP T0_ISR

ORG 0013H
LJMP INTR1_ISR

MAIN:
SETB EA
SETB EX1
SETB IT1
SETB EX0
SETB IT0
SETB ET0

CLR F0

HIGH_BYTE EQU 30H
```

```
MID_BYTE  EQU 31H
LOW_BYTE  EQU 32H

MOV TH1,#04CH
MOV TL1,#000H
SETB TR1

ACALL LCD_INIT
MOV A,#080H
ACALL LCD_CMD

CLR BLOWER_ENB
SETB MUL_EN
CLR ENA
SETB REV
SETB LED_R
CLR 20H.0
CLR VW2

mov dptr,#STR_menu
ACALL PRINT

;Waiting until a car is detected at stage 1
JB S1,$


;a car is detected
ACALL LCD_CLR
MOV DPTR,#MYSTRING
ACALL PRINT

;wait 3 s
ACALL DELAY_3S

;;;; CHECK WEIGHT
LOOP_WEIGHT:
ACALL LCD_CLR
mov dptr,#STR_WEIGHT
ACALL PRINT

MOV A,#0C0H
ACALL LCD_CMD

SETB SCK
CLR SCK

MOV R7,#50
WAIT_START:ACALL DELAY_1MS
DJNZ R7, WAIT_START

MOV R7,#25
SETUP:
SETB SCK
NOP
CLR SCK
NOP
DJNZ R7, SETUP
```

```
WAIT_READY:
JB DT, WAIT_READY

;read 24 bits
MOV R7, #24
CLR A
MOV HIGH_BYTE, A
MOV MID_BYTE, A
MOV LOW_BYTE, A

READ_24_BITS:
;pulse clock
SETB SCK
NOP
NOP

;read bit
MOV C, DT
MOV A, LOW_BYTE
RLC A
MOV LOW_BYTE, A

MOV A, MID_BYTE
RLC A
MOV MID_BYTE, A

MOV A, HIGH_BYTE
RLC A
MOV HIGH_BYTE, A

;end pulse
CLR SCK
NOP
NOP

DJNZ R7, READ_24_BITS

;one more pulse for next read
SETB SCK
NOP
CLR SCK

;only lower byte is considered for simplicity
MOV A,LOW_BYTE
MOV B,#100
DIV AB
MOV R0,A
MOV A,B
MOV B,#10
DIV AB
MOV R1,A
MOV R2,A

;write weight
MOV A,R0
ADD A,#030H
```

```
ACALL LCD_DATA_WRITE
MOV A,R1
ADD A,#030H
ACALL LCD_DATA_WRITE
MOV A,R2
ADD A,#030H
ACALL LCD_DATA_WRITE

ACALL DELAY_100MS
ACALL DELAY_100MS

JNB RST_BUT, MOVE_NEXT
SJMP LOOP_WEIGHT


;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

MOVE_NEXT:
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#MYSTRING1
ACALL PRINT


;waiting for 5 seconds
MOV R2,#5
ACALL START_TIMER

JNB 20H.0,$
;CLR LED_G

;running conveyor until it goes to stage 2
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#MYSTRING2
ACALL PRINT
SETB IN1
CLR IN2
GO:ACALL RUN_CONVEYOR
JNB S2,SKIP
SJMP GO

;starting water rinsing and continuing for 15 seconds
SKIP:
CLR MUL_EN

MOV R2,#15
ACALL START_TIMER
;SETB IN1_WASH
;CLR IN2_WASH
CLR MUL_A
CLR MUL_B
ACALL LCD_CLR
MOV DPTR,#STRING_3
ACALL PRINT
```

```
CHK1:
JB LIMIT,CHK2
ACALL DELAY_1MS
JNB LIMIT,CHK
SJMP CHK2

CHK:CPL MUL_A
CPL MUL_B
CHK2:
JNB 20H.0,CHK1


SETB MUL_EN
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

;soap spraying for 15 seconds

CLR MUL_EN
SETB MUL_A
CLR MUL_B
MOV R2,#15
ACALL START_TIMER
ACALL LCD_CLR
MOV DPTR,#STRING_4
ACALL PRINT

JNB 20H.0,$
SETB MUL_EN



;running conveyor until car is detected at stage 3
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#MYSTRING2
ACALL PRINT
SETB IN1
CLR IN2
GO1:ACALL RUN_CONVEYOR
JNB S3,SKIP2
SJMP GO1


SJMP $
;STAGE 3
;starting brushing motors for 10 seconds
SKIP2:
OK:MOV R2,#10
ACALL START_TIMER
;////SETB BRUSH_HP
;CLR VW2
CLR MUL_EN
CLR MUL_A
SETB MUL_B
mov dptr,#STRING_5
```

```
ACALL LCD_CLR
MOV A,#080H
acall print

JNB 20H.0,$

;SETB VW2
SETB MUL_EN


MOV R2,#10
ACALL START_TIMER


SETB VW2
mov dptr,#STRING_hp
ACALL LCD_CLR
MOV A,#080H
acall print

JNB 20H.0,$

CLR  VW2

;running conveyor until car is detected at stage 4
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#MYSTRING2
ACALL PRINT
SETB IN1
CLR IN2
GO2:ACALL RUN_CONVEYOR
JNB S4,SKIP3
SJMP GO2

;run blower for 15 seconds or until the car moves out of the stage 4
SKIP3:
MOV R2,#15
ACALL START_TIMER
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#STRING_6
ACALL PRINT

X11:
JB S4,EXIT11
JB 20H.0,EXIT11
CLR BLOWER_ENB
MOV R6,#75
DJNZ R6,$
SETB BLOWER_ENB
MOV R6,#25
DJNZ R6,$
SJMP X11
```

```
;process is completed
EXIT11:
CLR BLOWER_ENB
ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#STRING_7
ACALL PRINT

;waiting after the process is completed
sjmp $
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;;;;;;

;pwm signal for conveyor - 0.25 D
RUN_CONVEYOR:SETB ENA
MOV R0,#25
DJNZ R0,$
CLR ENA
MOV R0,#75
DJNZ R0,$
RET



;subroutine for 3s delay
DELAY_3S:
MOV R5,#3
L2:MOV R4,#20
L1:MOV TMOD,#00010001B
MOV TH0,#04CH
MOV TL0,#000H
SETB TR0
JNB TF0,$
CLR TF0
DJNZ R4,L1
DJNZ R5,L2
RET

;subroutine to print on LCD
PRINT:
PUSH ACC
NEXT_CHAR:CLR A
MOVC A,@A+DPTR
JZ STRING_END
ACALL LCD_DATA_WRITE
INC DPTR
JMP NEXT_CHAR
STRING_END:POP ACC
RET

;delay for LCD
LCD_DELAY:
MOV R5,#2
LOOP1:
MOV R6,#255
DJNZ R6,$
```

```
DJNZ R5,LOOP1
RET

;clear LCD display
LCD_CLR:
MOV A,#01H
ACALL LCD_CMD
ACALL LCD_DELAY
RET

;initialize LCD diintr_1splay
LCD_INIT:
ACALL LCD_DELAY
MOV A,#038H
ACALL LCD_CMD
MOV A,#0CH
ACALL LCD_CMD
MOV A,#001H
ACALL LCD_CMD
MOV A,#06H
ACALL LCD_CMD
RET

;send commands to LCD
LCD_CMD:
MOV LCD_DATA, A
CLR LCD_RS
SETB LCD_EN
ACALL LCD_DELAY
CLR LCD_EN
RET

;send data to print on LCD
LCD_DATA_WRITE:
MOV LCD_DATA, A
SETB LCD_RS
SETB LCD_EN
ACALL LCD_DELAY
CLR LCD_EN
RET

;interrupt to detect abnormal pressure
INTR1_ISR:

MOV C ,MUL_EN
MOV 20H.1,C
MOV C ,VW2
MOV 20H.2,C
MOV C,ENA
MOV 20H.3,C
MOV C ,BLOWER_ENB
MOV 20H.4,C

CLR LED_R

SETB MUL_EN
CLR VW2
```

```
CLR ENA
CLR BLOWER_ENB

ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
mov dptr,#STR_P
ACALL PRINT

POINT:JB RST_BUT,$
ACALL DELAY_1MS
JNB RST_BUT,OUT
SJMP POINT

OUT:
MOV C ,20H.1H
MOV MUL_EN,C
MOV C ,20H.2
MOV VW2,C
MOV C ,20H.3
MOV ENA,C
MOV C ,20H.4
MOV BLOWER_ENB,C

SETB LED_R

ACALL LCD_CLR
MOV A,#080H
ACALL LCD_CMD
mov dptr,#STR_RESUME
ACALL PRINT
RETI

;interrupt to handle emergency stop and moving the conveyor
;backward and forward manually during the emergency stop
INTR0_ISR:
CLR BLOWER_ENB
CLR ENA
CLR IN1
CLR IN2
CLR LED_R

MOV A,#080H
ACALL LCD_CMD
mov dptr,#STR_EMERG
ACALL PRINT
LOOP2:
JB RST_BUT,PROCEED
MOV A,#080H
ACALL LCD_CMD
mov dptr,#STR_RESET
ACALL PRINT

ACALL DELAY_3S
CLR RST_MCU
MOV R7,#60
DJNZ R7,$
```

```
SETB RST_MCU
RETI

PROCEED:JNB FWD,MOVE_FWD
JNB REV,MOVE_REV
JB FWD,OFF
JB REV,OFF
SJMP LOOP2
MOVE_FWD:
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#STR_FWD
ACALL PRINT
SETB ENA
SETB IN1
CLR IN2
SJMP LOOP2
MOVE_REV:
MOV A,#080H
ACALL LCD_CMD
MOV DPTR,#STR_REV
ACALL PRINT
SETB ENA
CLR IN1
SETB IN2
SJMP LOOP2
OFF:CLR ENA
CLR IN1
CLR IN2
MOV A,#080H
ACALL LCD_CMD
mov dptr,#STR_EMERG
ACALL PRINT
MOV A,#0C0H
ACALL LCD_CMD
MOV DPTR,#STR_BLANK
ACALL PRINT
SJMP LOOP2

RETI

;subroutine to start timer
;before calling this, the required number of seconds should be
stored in R2
START_TIMER:
MOV R0,#0
MOV R1,#0
SETB ET0
CLR 20H.0

MOV TMOD,#00010001B
MOV TH0,#04CH
MOV TL0,#000H
SETB TR0
RET

;using timer interrupt to measure time
```

```
T0_ISR:
CLR TR0
INC R0
MOV A,R0
CJNE A,#20,EXIT
MOV R0,#0
INC R1
MOV A,R1
MOV B,R2
CJNE A,B,EXIT
MOV R0,#0
MOV R1,#0

SETB 20H.0
CLR ET0

EXIT:
CLR TF0
MOV TH0,#04CH
MOV TL0,#000H
SETB TR0
RETI

DELAY_1MS:
MOV R6, #250
DELAY_1MS_LOOP:
NOP
NOP
DJNZ R6, DELAY_1MS_LOOP
RET


DELAY_100MS:
MOV R7, #200
DELAY_1:
MOV R6, #250
DELAY_2:
NOP
NOP
DJNZ R6, DELAY_2
DJNZ R7, DELAY_1
RET

;storing strings to print on LCD
MYSTRING: DB 'Vehicle detected',0
MYSTRING1:DB 'Washing started',0
MYSTRING2:DB 'Conveyor moving',0
STRING_3:DB 'Rinsing vehicle',0
STRING_4:DB 'Applying soap',0
STRING_5:DB 'Brushing and HP Wash',0
STRING_HP:DB 'HP Washing',0
STRING_6:DB 'Drying the vehicle',0
STRING_7:DB 'Process Complete',0
STR_EMERG:DB 'Emergency Stopped',0
STR_FWD: DB 'Move Forward    ',0
STR_BLANK: DB '                ',0
STR_REV: DB 'Move Backward   ',0
```

```
STR_RESET: DB 'SYSTEM RESET          ',0
STR_WEIGHT: DB 'Weight :          ',0
STR_menu: DB '### ACWS ###    ',0
STR_P: DB 'LP DETECTED',0
STR_RESUME: DB 'SYSTEM RESUMING',0
```

[Q6] Identify the design issues and the necessary improvements of your design.

In order to measure the weight, a load cell is using with the HX711 ADC module. The 8051 is having an 8 bit data bus. But the HX711 outputs a 24 bit pattern for weight. Because of that it provides a measurement with high resolution. But as 8051 is having 8 bit data bus, it requires at least 3 registers to store those bytes. In order to represent the weight, this binary data has to be converted in to decimal. It is somewhat complex to perform that operation with 8051 as it is capable only to do 8 bit arithmetic operations but in modern micro controllers like ESP32 have 32 bit data bus so that the conversions can be done more conveniently. Also 8051 is not having any internal ADC, it is the reason for using this external ADC.

Also for critical application like a car washing system, the clock speed of 8051 will not be sufficient as it has a clock speed of 11.592 MHz. But modern microcontrollers such as ESP32 have clock speeds of around 240 MHz which means they can process faster. It will be helpful to make readings more accurate like by taking several measurements within a small time and getting an average of them. But it will take more time to do that process on 8051 and it will add delays for the process.

Another issue faced during the designing of this project was the limited number of general purpose pins available on 8051. As a solution for that we can connect two 8051 MCUs together through serial communication as master and slave or we can use port expanders which uses I2C/SPI. In this project, two 1 to 4 decoder ICs are used to control multiple actuators by using a minimum number of pins. Then we can increase the number of pins available for our purposes. Also there's a limited number of interrupts in 8051, for time critical functions, it would require more interrupt pins so that we can perform more real time tasks simultaneously.

When implementing this in real world, noise may be a major issue for sensors like the load cells. Therefore quality equipment should be used for sensors in order to minimize the effect of noise as much as possible. Here the emergency stop button works on software, if the MCU fails, it wouldn't be able to detect that signal and stop the system. Therefore it is better to use a hardware based system independent of software for emergency stop function as it will immediately disconnect the actuators from supply power regardless of software program.

If a modern microcontroller such as ESP32 is used for making this project, it can be developed adding many more features with better performance. As an example ESP32 is having WIFI and Bluetooth connectivity, therefore a web application can be designed to control and monitor the whole ACWS from one interface. Also it would allow parallel processing as well which will allow us to carry more than one task simultaneously as they have multiple cores. Also data can be collected through a system like this and those data can be analyzed ad improve the system to obtain its maximum.

[Q7] Is it possible to implement your design on an ASIC (Application Specific Integrated Circuit)? Justify your answer?

An application Specific Integrated Circuit is a type of microchip designed and manufactured for a specific application or a purpose. They are custom designed for one task and have a fixed functionality. It is possible to implement the design of this ACWS as an ASIC. A circuit can be designed including the logic to get readings from sensors, and to generate control signals based on those inputs to run actuators. Also safety implementations such as emergency stop, overload motor stop, etc. But this is a kind of system which requires persistent and continuous upgrades. If this was designed as an ASIC, it would be hard to make changes to the program and it would add more costs to build it from scratch again after any changes to the program. Therefore it wouldn't be a much economical and feasible option to build ACWS as an ASIC. Instead of that using a microcontroller will be a better option as the program can be modified as per our requirement and can be implemented in no time without building it from scratch again.

[Q8] List the ethical, social issues/considerations of your design

This system handles vehicles, therefore it requires more powerful motors which are capable of handling that much of loads. Therefore if any kind of fault or a mistake happened, it will cause serious damages to people who are working there. Even they may lost their lives or be permanently disabled. Therefore redundant safety implementations should be used in the design in order to avoid accidents that can happen in any manner.

Also a car washing uses many chemicals and water. If it is an area where water is scare, using huge amount of water for this kind of application would be a problem to general people who

consume water to drink. Another serious issue is the way of removing the waste products. They may contain chemical agents which are poisonous if added to the environment. Therefore there should be a proper waste management plan in order to avoid harming the environment from the bi products. Using a water recycling method will help to use the resources efficiently.

This system will require a considerable energy to power the motors, conveyors with heavy loads. If the electricity grid in that area is not capable providing that much of power it will affect for the other people's energy requirements living around that area. Therefore it is better to go for renewable energy sources like on grid or off grid solar energy.

Another major impact that this system will do on the society is that job displacement. If this process which is going to do by this system was done manually by people, it would require a number of persons to do that. But this system will replace all of those persons jobs. It will be a critical social problem arising difficulty for them to live due to cut down of an income. A better solution for this is they can be given a training on how to monitor this system and how should they act in an emergency or during a failure. Although this is an automated system, human intervention is important during a failure or an emergency. Anyway it will affect for the displacement of jobs of a number of persons.